



Модуль 2. Асинхронное программирование: теория и практика

Введение в модуль

В Модуле 1 мы изучили механизмы Python, лежащие в основе асинхронности: итераторы сохраняют состояние, генераторы приостанавливают выполнение, менеджеры контекста управляют жизненным циклом ресурсов. Теперь мы поднимаемся на уровень выше — к самой **кооперативной многозадачности** и её сердцу, **событийному циклу**.

Этот модуль — самый плотный теоретически. Мы разберём:

- почему **конкурентность** не равна **параллелизму**;
- как устроен **event loop** от системных вызовов до очереди задач;
- что такое **корутина** с точки зрения теории вычислений;
- как управлять множеством корутин и синхронизировать их;
- куда девать **CPU-bound** задачи в однопоточном цикле событий.

2.1. Конкурентность, параллелизм, кооперативная многозадачность

2.1.1. Concurrency ≠ Parallelism

Это различие фундаментально. Его непонимание — источник большинства ошибок при проектировании асинхронных систем.

Конкурентность (concurrency) — это свойство системы обрабатывать **несколько задач в перекрывающиеся периоды времени**. Задачи чередуются: пока одна ждёт, другая выполняется. В один момент реально работает только одна.

Параллелизм (parallelism) — это свойство системы **одновременно** выполнять несколько задач на разных вычислительных единицах (ядрах CPU, GPU-стриминг-мультипроцессорах).

Аналогия: представьте повара на кухне.

- **Параллелизм** — два повара готовят два блюда одновременно на двух плитах.
- **Конкурентность** — один повар готовит два блюда: поставил воду на плиту, пока она закипает — нарезал овощи, пока овощи тушатся — помешал суп. Блюда готовятся «одновременно» с точки зрения заказчика, но повар в каждый момент делает только одно действие.

Асинхронный Python — это **конкурентность без параллелизма** (если не считать `run_in_executor`). Один поток, один event loop, тысячи корутин, которые по очереди получают управление.

2.1.2. Вытесняющая vs кооперативная многозадачность

Операционная система управляет процессами и потоками через **вытесняющую многозадачность (preemptive multitasking)**. Она работает так:

- Каждому потоку выделяется **квант времени** (time slice), например 10 мс.
- По истечении кванта таймер прерывания заставляет CPU переключиться на другой поток.
- Поток не контролирует, когда его прервут. Это решает ОС.

Кооперативная многозадачность (cooperative multitasking) работает иначе:

- Задача сама решает, когда уступить управление.
- Она делает это в **точках ожидания**: «я сейчас буду ждать ответа от базы данных, пока жду — займись другими».
- Если задача не уступает управление добровольно, она **блокирует весь цикл**.

В Python `asyncio` — кооперативная модель. Ключевое слово `await` — это именно та точка, где корутина говорит: «я уступаю управление, верните мне его, когда результат будет готов».

2.1.3. Почему кооперативная модель эффективна для I/O?

Потому что в веб-приложениях задачи проводят **большую часть времени в ожидании**:

Операция	Время (порядок)
CPU: сложение двух чисел	1 нс
CPU: обращение к L1 кэшу	1 нс
CPU: обращение к оперативной памяти	100 нс
Диск (SSD): чтение 4 КБ	10 мкс
Сеть (внутри дата-центра): RTT	500 мкс
Сеть (межконтинентальная): RTT	150 мс
Человек: клик мышью	100-300 мс

Когда сервер ждёт ответа от базы данных (500 мкс–5 мс), CPU мог бы выполнить **миллионы** арифметических операций. Кооперативная многозадачность позволяет не тратить это время впустую.

2.1.4. Исторический контекст: от callback-hell к сопрограммам

До появления `async / await` в Python 3.5 асинхронный код писался через **колбэки**. Это была эпоха `Twisted` и раннего `Tornado`:

```
In [ ]: # Псевдокод в духе Twisted
def handle_request(request):
    d = database.query("SELECT ...")
    d.addCallback(on_data_ready)
    d.addErrback(on_error)

def on_data_ready(result):
    d = cache.set("key", result)
    d.addCallback(on_cache_set)

def on_cache_set(_):
    return Response("OK")
```

Проблемы:

- **Callback Hell**: вложенность колбэков растёт экспоненциально.
- **Разрыв контекста**: исключения теряются между колбэками.

- **Сложность композиции:** нельзя написать `result1 + result2`, если оба результата приходят асинхронно.

CSP (Communicating Sequential Processes) — модель Тони Хоара, где процессы взаимодействуют через каналы. Она легла в основу Go (горутины + channels). Python пошёл другим путём.

Модель акторов — каждый «актор» (объект) имеет почтовый ящик и обрабатывает сообщения последовательно. Акторы не разделяют состояние. Это Erlang, Akka. Python не использует эту модель напрямую, но `asyncio` позволяет эмулировать акторов через задачи с очередями.

Python выбрал **сопрограммы (coroutines)** с явными точками уступки (`await`) — золотую середину между читаемостью и производительностью.

2.1.5. Математическая подоплека: конечные автоматы и машины состояний

Каждая корутина в `asyncio` — это **конечный автомат** (finite state machine, FSM), хотя и с бесконечным числом состояний из-за произвольных локальных переменных.

Формально, конечный автомат — это кортеж:

$$M = (Q, \Sigma, \delta, q_0, F)$$

где:

- Q — множество состояний,
- Σ — алфавит входных символов (событий),
- $\delta: Q \times \Sigma \rightarrow Q$ — функция перехода,
- q_0 — начальное состояние,
- F — множество финальных состояний.

Для корутины:

- Состояния: `CREATED`, `RUNNING`, `SUSPENDED`, `DONE`, `CANCELLED`.
- События: `await` (приостановка), `resolve future` (возобновление), `cancel` (отмена).
- Переходы: `RUNNING --await--> SUSPENDED --future resolved--> RUNNING --return--> DONE`.

Event loop — это **диспетчер**, который хранит множество таких автоматов и решает, какому из них передать управление в зависимости от внешних событий (готовность сокета, таймаут).

2.2. Событийный цикл (Event Loop)

2.2.1. Что такое event loop?

Event loop — это **бесконечный цикл**, который:

1. Проверяет, есть ли готовые к выполнению задачи (coroutines).
2. Если есть — передаёт управление следующей задаче.
3. Если нет — **засыпает**, дожидаясь внешнего события (данные пришли по сети, прошёл таймаут).
4. Просыпается, когда событие произошло, и возвращается к шагу 1.

Это **Reactor Pattern** — один из классических паттернов проектирования сетевых приложений.

In [4]: `import asyncio`

```
async def main():
    print("Hello")
    await asyncio.sleep(1)  # <- точка уступки управления
    print("World")

# Запуск event loop (в Jupyter не работает, т.к. он уже запустил event lo
# asyncio.run(main())

# Запуск для Jupyter
await main()
```

Hello
World

Что происходит под капотом `asyncio.run()` ?

1. Создаётся новый event loop.
2. Регистрируется корутина `main()` как задача (Task).
3. Запускается цикл: `while loop.is_running(): loop._run_once()`.
4. Когда `main()` завершится, цикл останавливается и закрывается.

2.2.2. Архитектура event loop

Event loop состоит из нескольких структур данных:

Структура	Назначение
Ready queue	Очередь корутин, готовых к выполнению прямо сейчас.
Scheduled queue	Куча (heap) корутин с таймаутами: <code>call_later</code> , <code>sleep</code> .
Selector	Механизм ОС для отслеживания готовности файловых дескрипторов (сокетов).

Алгоритм одной итерации цикла (упрощённо):

```
In [ ]: def run_once(self):
    # 1. Переместить просроченные scheduled-задачи в ready queue
    now = self.time()
    while scheduled and scheduled[0].when <= now:
        handle = heapq.heappop(scheduled)
        ready.append(handle)

    # 2. Выполнить все задачи из ready queue
    while ready:
        handle = ready.popleft()
        handle._run()

    # 3. Определить, сколько можно спать
    timeout = 0 if ready else (scheduled[0].when - now if scheduled else

    # 4. Дождаться событий от ОС (сокеты, таймауты)
    events = selector.select(timeout)

    # 5. Пробудить корутины, ожидающие эти события
    for key, mask in events:
        callback = key.data
        callback()
```

2.2.3. Селекторы: select, poll, epoll, kqueue

Event loop не опрашивает сокеты в цикле («busy waiting») — это сожгло бы CPU. Вместо этого он делегирует ОС через системные вызовы:

select (BSD, 1983) — самый старый. Передаёт ОС **массивы** файловых дескрипторов. Ограничения:

- Максимальный размер массива: `FD_SETSIZE` = 1024 (на многих системах).

- При каждом вызове ядро копирует массивы из пользовательского пространства в ядро и обратно: $O(n)$.

poll (System V, 1997) — убирает лимит 1024, но оставляет $O(n)$: ядро проверяет каждый дескриптор в списке.

epoll (Linux, 2002) — революция. Работает по принципу **готового списка**:

- Регистрация дескриптора: `epoll_ctl(EPOLL_CTL_ADD)` — $O(1)$.
- Ожидание событий: `epoll_wait` — возвращает **только** готовые дескрипторы, $O(k)$, где k — число готовых.
- Не нужно передавать весь массив при каждом вызове.

kqueue (FreeBSD, macOS) — аналог `epoll` для BSD-систем.

Python's `asyncio` использует `selectors.DefaultSelector`, который автоматически выбирает лучший доступный механизм (`epoll` на Linux, `kqueue` на macOS, `select` как fallback).

2.2.4. Почему `time.sleep(10)` убивает производительность?

Рассмотрим два примера:

```
In [ ]: # Вариант А: асинхронный сон
        async def handler():
            await asyncio.sleep(10) # <- уступает управление
            return "done"

        # Вариант Б: синхронный сон
        async def handler():
            time.sleep(10) # <- блокирует весь поток!
            return "done"
```

В варианте А `asyncio.sleep(10)` не засыпает реально. Он регистрирует в event loop: «разбуди меня через 10 секунд» и **уступает управление**. За эти 10 секунд цикл обработает тысячи других запросов.

В варианте Б `time.sleep(10)` — **системный вызов**, который блокирует ОС-поток. Event loop работает в одном потоке. Поток засыпает. Все корутины в цикле замирают. Сервер перестаёт отвечать.

Это самая распространённая ошибка в асинхронном Python:

вызвать синхронную блокирующую функцию внутри `async def`.

2.2.5. API event loop: создание и управление

```
In [ ]: import asyncio

# Получить текущий цикл (внутри корутины)
loop = asyncio.get_running_loop()

# Получить или создать цикл (устаревший способ, до Python 3.10)
loop = asyncio.get_event_loop()

# Запустить корутину до завершения
asyncio.run(main()) # предпочтительный способ

# Низкоуровневый запуск
loop = asyncio.new_event_loop()
asyncio.set_event_loop(loop)
try:
    loop.run_until_complete(main())
finally:
    loop.close()
```

Важно: `asyncio.run()` — высокоуровневая функция, предназначенная для точки входа. Она создаёт цикл, запускает корутину, останавливает цикл. Внутри уже запущенного цикла вызывать `asyncio.run()` нельзя — будет ошибка.

2.2.6. Математическая подоплека: теория очередей

Event loop — это система массового обслуживания (СМО). Формально:

- **Источник заявок:** входящие HTTP-запросы, таймауты, события сокетов.
- **Очередь:** ready queue + scheduled queue.
- **Сервер:** один «прибор» (одно ядро CPU, один поток), обрабатывающий заявки.
- **Дисциплина обслуживания:** FIFO (First In, First Out) для ready queue.

Это модель **M/M/1** в теории очередей, если считать, что поступление заявок — пуассоновский процесс, а время обслуживания — экспоненциальное распределение.

Для M/M/1 известны формулы:

- Средняя длина очереди: $L_q = \frac{\rho^2}{1-\rho}$, где $\rho = \frac{\lambda}{\mu}$ — коэффициент загрузки.
- Среднее время ожидания: $W_q = \frac{\rho}{\mu(1-\rho)}$.

Когда $\rho \rightarrow 1$ (система близка к полной загрузке), очередь и время ожидания стремятся к бесконечности. Это объясняет, почему асинхронный сервер «падает» не линейно, а катастрофически при перегрузке — эффект **лавинной деградации**.

2.3. Корутины: `async` / `await`

2.3.1. Корутина как объект

Когда вы пишете:

```
In [5]: async def fetch_data(url):
        response = await aiohttp.get(url)
        return response.json()
```

функция `fetch_data` **не выполняется**. Она возвращает объект **корутины**:

```
In [6]: coro = fetch_data("https://api.example.com")
        print(type(coro)) # <class 'coroutine'>
        print(coro)      # <coroutine object fetch_data at 0x...>
```

```
<class 'coroutine'>
<coroutine object fetch_data at 0x00000222BF9E01E0>
```

Корутина — это **объект, хранящий состояние** приостановленной функции: локальные переменные, стек вызовов, точка, на которой остановились. Это прямое развитие генераторов из Модуля 1.

2.3.2. Три ключевых объекта: Coroutine, Future, Task

Объект	Что это	Аналогия
Coroutine	Функция с <code>async def</code> , ещё не запущенная.	Черновик задачи.
Future	Контейнер для результата, который появится в будущем.	Обещание (promise): «когда-нибудь здесь будет значение».
Task	Корутина, обёрнутая в задачу и поставленная в очередь event loop.	Задача в планировщике.

Future — низкоуровневый примитив. Он имеет состояние и результат:

```
In [7]: future = asyncio.Future()
print(future.done())    # False
future.set_result(42)
print(future.result())  # 42
```

```
False
42
```

Обычно вы не создаёте Future вручную — они создаются внутри `asyncio` при `await`'е на I/O-операциях.

Task — это Future + Coroutine. Task запускает корутину в event loop и позволяет отслеживать её выполнение:

```
In [8]: async def say_hello():
        await asyncio.sleep(1)
        return "Hello"

# Создание задачи
task = asyncio.create_task(say_hello())
# Теперь say_hello выполняется "в фоне", конкурентно с текущей корутиной
```

2.3.3. Ключевое слово `await`: точка уступки управления

`await` — это **единственная** точка, где корутина может уступить управление event loop. Синтаксически:

```
In [ ]: result = await some_awaitable
```

Что происходит:

1. Проверяется, является ли `some_awaitable` **awaitable** (имеет метод `__await__`).
2. Если объект — корутина, она запускается (или возобновляется).
3. Текущая корутина **приостанавливается** и снимается с выполнения.
4. Event loop получает управление и может запустить другую задачу.
5. Когда `some_awaitable` завершится, текущая корутина **возобновляется** с точки после `await`, и результат присваивается переменной.

Критически важно: `await` — это не «ждать, ничего не делая». Это «уступить управление, чтобы другие поработали».

2.3.4. Состояния корутины / задачи

```
In [9]: import asyncio

async def lifecycle_demo():
    print("RUNNING")
    await asyncio.sleep(0) # уступаем, но сразу возвращаемся
    print("BACK")

task = asyncio.create_task(lifecycle_demo())

# Состояния, которые может иметь задача:
# PENDING — создана, но ещё не запущена
# RUNNING — выполняется прямо сейчас
# DONE — завершена (успешно или с исключением)
# CANCELLED — была отменена
```

RUNNING
BACK

Проверка состояний:

```
In [10... print(task.done()) # False, пока не завершится
print(task.cancelled()) # False, если не отменяли
print(task.result()) # Блокирует, пока задача не done; потом возвраща
```

True
False
None

Если задача завершилась с исключением, `task.result()` выбросит это исключение.

2.3.5. `await` vs `yield from`

В Python 3.4 корутины создавались через декоратор `@asyncio.coroutine` и `yield from`:

```
In [ ]: @asyncio.coroutine
def old_style():
    result = yield from asyncio.sleep(1)
    return result
```

В Python 3.5+ `async def` и `await` — это **синтаксический сахар** над тем же механизмом, но с важными ограничениями:

- `await` работает только с awaitable-объектами (корутины, задачи, фьючерсы).

- `yield from` работал с любыми итераторами.

Под капотом `await` вызывает `__await__()`, который возвращает итератор. То есть `await` — это `yield from` в специализированной форме.

2.3.6. Математическая подоплека: сопрограммы в теории вычислений

В теории языков программирования **сопрограмма (coroutine)** — это обобщение подпрограммы (обычной функции). Различия:

Подпрограмма	Сопрограмма
Единая точка входа.	Множественные точки входа (возобновление).
Выполняется от начала до конца.	Приостанавливается и возобновляется.
Завершается с <code>return</code> .	Завершается с <code>return</code> или <code>yield</code> .
Вызов подпрограммы — стек растёт.	Сопрограммы могут передавать управление друг другу без стекового вложения.

Формально, сопрограмма — это **continuation** (продолжение). Continuation — это абстрактное представление «остатка вычисления». Когда корутина встречает `await`, она захватывает свою continuation и передаёт её event loop: «вызови меня снова, когда будут готовы данные».

Это соответствует **Continuation-Passing Style (CPS)** — стилю программирования, где вместо возврата значения функция передаёт результат в callback-continuation. `async / await` — это синтаксический сахар над CPS, который делает код читаемым, сохраняя семантику.

2.4. Параллельное выполнение корутин

2.4.1. `asyncio.gather()`: параллельный запуск

`gather` принимает несколько awaitable и запускает их **конкурентно**:

```
In [12... import asyncio

async def fetch(url, delay):
    await asyncio.sleep(delay)
```

```

    return f"Data from {url}"

async def main():
    results = await asyncio.gather(
        fetch("A", 2),
        fetch("B", 1),
        fetch("C", 3),
    )
    print(results)  # ['Data from A', 'Data from B', 'Data from C']

# Запуск в Jupyter не работает, т.к. он уже запустил event loop для своей
# asyncio.run(main())

# Запуск для Jupyter
await main()

['Data from A', 'Data from B', 'Data from C']

```

In [17...

```

async def long_operation(name, delay):
    print(f"{name}: начал")
    await asyncio.sleep(delay)
    print(f"{name}: закончил")
    return f"{name}: {delay} секунд"

async def main():

    task1 = asyncio.create_task(long_operation("A", 3))
    task2 = asyncio.create_task(long_operation("B", 1))
    task3 = asyncio.create_task(long_operation("C", 2))

    print("Все задачи созданы, ждем завершения...")

    results = await asyncio.gather(task1, task2, task3)
    print(f"Результаты: {results}")

await main()

```

Все задачи созданы, ждем завершения...

A: начал

B: начал

C: начал

B: закончил

C: закончил

A: закончил

Результаты: ['A: 3 секунд', 'B: 1 секунд', 'C: 2 секунд']

Важные детали:

- `gather` возвращает результаты **в том же порядке**, в каком переданы аргументы, независимо от порядка завершения.
- Если одна из корутин выбросит исключение, `gather` по умолчанию **сразу** выбросит её. Остальные задачи продолжат выполняться в

фоне, но их результаты будут потеряны.

- Опция `return_exceptions=True` превращает исключения в возвращаемые значения:

```
In [ ]: results = await asyncio.gather(
    fetch("A", 1),
    failing_fetch("B"), # выбросит Exception
    return_exceptions=True
)
# results = ['Data from A', ValueError(...)]
```

2.4.2. `asyncio.wait()`: тонкое управление

`wait` даёт больше контроля, чем `gather`:

```
In [ ]: done, pending = await asyncio.wait(
    [task1, task2, task3],
    return_when=asyncio.FIRST_COMPLETED # или ALL_COMPLETED, FIRST_EXCEP
)
```

Параметр <code>return_when</code>	Когда возвращает управление
<code>ALL_COMPLETED</code>	Когда все задачи завершены (по умолчанию).
<code>FIRST_COMPLETED</code>	Когда хотя бы одна задача завершена.
<code>FIRST_EXCEPTION</code>	Когда хотя бы одна задача завершилась с исключением.

```
In [22... async def main():
    tasks = [
        asyncio.create_task(fetch("A", 2)),
        asyncio.create_task(fetch("B", 1)),
    ]

    done, pending = await asyncio.wait(tasks, return_when=asyncio.FIRST_C
    print(f"Готово: {len(done)}, В ожидании: {len(pending)}")
    print(f"done after .wait(tasks, return_when=asyncio.FIRST_COMPLETED):
    # Не забываем дождаться оставшихся
    if pending:
        await asyncio.wait(pending)

await main()
```

Готово: 1, В ожидании: 1

done after .wait(tasks, return_when=asyncio.FIRST_COMPLETED):

```
{<Task finished name='Task-141' coro=<fetch() done, defined at C:\Users\i
gory\AppData\Local\Temp\ipykernel_14188\3805592031.py:3> result='Data fro
m B'>}
```

2.4.3. `asyncio.as_completed()`: потоковая обработка

Если нужно обрабатывать результаты по мере готовности, не дожидаясь всех:

```
In [27... import time

async def fetch(url, delay):
    await asyncio.sleep(delay)
    return f"Data from {url}"

async def main():
    coros = [fetch(f"url_{i}", i) for i in [3, 1, 2]]

    for coro in asyncio.as_completed(coros):
        print("check")
        result = await coro # возвращает результат первой готовой, потом
        print(time.perf_counter(), result)
        # Вывод: url_1 (через 1с), url_2 (через 2с), url_3 (через 3с)

await main()

check
20721.3276263 Data from url_1
check
20722.3361115 Data from url_2
check
20723.3301538 Data from url_3
```

`as_completed` возвращает итератор корутин. Каждый `await` на очередной элемент даёт результат следующей завершившейся задачи.

2.4.4. `asyncio.create_task()`: фоновые задачи

`create_task` — основной способ запустить корутину «в фоне»:

```
In [ ]: async def background_worker():
        while True:
            await process_queue()
            await asyncio.sleep(1)

async def main():
    # Запускаем воркер в фоне
    task = asyncio.create_task(background_worker())

    # Одновременно обрабатываем HTTP-запросы
    await start_server()
```

```
# При завершении main отменяем фоновую задачу
task.cancel()
try:
    await task
except asyncio.CancelledError:
    print("Фоновая задача отменена")
```

Важно: в Python 3.11+ `asyncio.CancelledError` наследуется от `BaseException`, а не `Exception`. Это сделано для того, чтобы `except Exception` не перехватывал отмену случайно.

2.4.5. Отмена задач и таймауты

Отмена:

```
In [30... task = asyncio.create_task(long_operation("A", 2))

# Отмена
task.cancel()

try:
    await task
except asyncio.CancelledError:
    print("Задача была отменена")
```

Задача была отменена

Когда вызывается `task.cancel()`, в корутину **вбрасывается** `CancelledError` в ближайшей точке `await`. Если корутина перехватывает `CancelledError` и не пробрасывает его дальше — отмена будет проигнорирована!

Это антипаттерн:

```
In [ ]: # ПЛОХО: подавление отмены
async def bad():
    try:
        await asyncio.sleep(10)
    except asyncio.CancelledError:
        print("Отмена? Не слышал")
        # Не пробрасываем! Задача не завершится.
```

Правильно:

```
In [ ]: # ХОРОШО: пробрасываем отмену
async def good():
    try:
        await asyncio.sleep(10)
    except asyncio.CancelledError:
```



```
cleanup()
raise # <- обязательно пробрасываем
```

Таймауты:

```
In [ ]: # Python 3.11+
        async with asyncio.timeout(5):
            await slow_operation()

        # До 3.11
        await asyncio.wait_for(slow_operation(), timeout=5.0)
```

`asyncio.timeout` создаёт контекстный менеджер, который отменяет всё внутри блока, если время истекло.

2.5. Синхронизация в асинхронном коде

Кооперативная многозадача даёт иллюзию изоляции, но корутины разделяют **один поток** и **одну память**. Если две корутины одновременно (в перекрывающиеся моменты) модифицируют разделяемую структуру — состояние гонки (race condition) возможно.

2.5.1. `asyncio.Lock`: взаимное исключение

```
In [ ]: lock = asyncio.Lock()
        shared_counter = 0

        async def increment():
            global shared_counter
            async with lock: # <- только одна корутина может войти
                current = shared_counter
                await asyncio.sleep(0) # имитация "тяжёлой" операции
                shared_counter = current + 1
```

Без `lock` возможна интерливинг:

Время	Корутина А	Корутина В
t1	current = 0	
t2	await sleep(0) -> уступка	
t3		current = 0
t4		shared_counter = 1
t5	shared_counter = 1	

Результат: два инкремента, но `shared_counter == 1` вместо 2.

`asyncio.Lock` гарантирует, что критическая секция выполняется атомарно относительно других корутин.

2.5.2. `asyncio.Semaphore`: ограничение параллелизма

`Semaphore` — счётчик разрешений. Позволяет одновременно выполнять не более N корутин:

```
In [ ]: # Не более 5 одновременных запросов к внешнему API
        semaphore = asyncio.Semaphore(5)

        async def fetch_limited(url):
            async with semaphore:
                return await aiohttp.get(url)

        # Запускаем 100 запросов, но одновременно выполняется только 5
        tasks = [asyncio.create_task(fetch_limited(u)) for u in urls]
        results = await asyncio.gather(*tasks)
```

`BoundedSemaphore` отличается тем, что не позволяет освободить больше разрешений, чем было захвачено (защита от багов).

2.5.3. `asyncio.Event`, `Condition`, `Barrier`

Event — флаг, который могут ждать множество корутин:

```
In [34... event = asyncio.Event()

        async def waiter(name):
            print(f"{name}: жду события...")
            await event.wait()
            print(f"{name}: событие произошло!")

        async def setter():
            await asyncio.sleep(2)
            event.set() # <- все ожидающие пробуждаются

        async def main():
            await asyncio.gather(
                waiter("A"), waiter("B"), setter()
            )

        await main()
```

A: жду события...
B: жду события...
A: событие произошло!
B: событие произошло!

Condition — Event + Lock. Позволяет ждать условия с атомарной проверкой:

```
In [ ]: # Создаем условие (внутри есть Lock)
condition = asyncio.Condition()

# Основные методы:
# condition.wait()      - ждать сигнала (отпускает lock)
# condition.notify()    - разбудить ОДНОГО ожидающего
# condition.notify_all() - разбудить ВСЕХ ожидающих

# Использование:
async with condition:
    # критическая секция (lock захвачен)
    while not check_condition():
        await condition.wait() # отпускает lock и спит
    # здесь условие выполнено
```

```
In [ ]: condition = asyncio.Condition()
queue = []

async def consumer():
    async with condition:          # захват lock
        while not queue:          # проверка условия
            await condition.wait() # если нет, то отпускает lock, ждёт n
            # проснулись - lock снова захвачен
        item = queue.pop(0)
        print(f"Обработано: {item}")

async def producer():
    async with condition:          # захват lock
        queue.append("data")       # изменяем состояние
        condition.notify_all()     # будим всех ожидающих
```

```
In [35... from collections import deque

class AsyncQueue:
    def __init__(self, maxsize=10):
        self._queue = deque()
        self._maxsize = maxsize
        self._condition = asyncio.Condition()

    async def put(self, item):
        async with self._condition:
            # Ждем, пока есть место
            while len(self._queue) >= self._maxsize:
                print(f"Очередь полна, ждем... (size={len(self._queue)})"
                await self._condition.wait()
```

```

        # Добавляем элемент
        self._queue.append(item)
        print(f"Добавлено: {item}, size={len(self._queue)}")

        # Будим одного потребителя (или всех)
        self._condition.notify()

    async def get(self):
        async with self._condition:
            # Ждем, пока есть элементы
            while not self._queue:
                print(f"Очередь пуста, ждем...")
                await self._condition.wait()

            # Забираем элемент
            item = self._queue.popleft()
            print(f"Забрано: {item}, size={len(self._queue)}")

            # Будим производителя (освободилось место)
            self._condition.notify()
            return item

async def producer(queue, name, count):
    for i in range(count):
        item = f"{name}-{i}"
        await queue.put(item)
        await asyncio.sleep(0.5)
    print(f"Производитель {name} завершил работу")

async def consumer(queue, name, count):
    for _ in range(count):
        item = await queue.get()
        print(f"Потребитель {name} обработал: {item}")
        await asyncio.sleep(1) # имитация обработки
    print(f"Потребитель {name} завершил работу")

async def main():
    queue = AsyncQueue(maxsize=3)

    await asyncio.gather(
        producer(queue, "P1", 5),
        producer(queue, "P2", 5),
        consumer(queue, "C1", 5),
        consumer(queue, "C2", 5),
    )

await main()

```

Добавлено: P1-0, size=1
 Добавлено: P2-0, size=2
 Забрано: P1-0, size=1
 Потребитель C1 обработал: P1-0
 Забрано: P2-0, size=0
 Потребитель C2 обработал: P2-0
 Добавлено: P1-1, size=1
 Добавлено: P2-1, size=2
 Забрано: P1-1, size=1
 Потребитель C1 обработал: P1-1
 Забрано: P2-1, size=0
 Потребитель C2 обработал: P2-1
 Добавлено: P1-2, size=1
 Добавлено: P2-2, size=2
 Добавлено: P1-3, size=3
 Очередь полна, ждем... (size=3)
 Забрано: P1-2, size=2
 Потребитель C1 обработал: P1-2
 Забрано: P2-2, size=1
 Потребитель C2 обработал: P2-2
 Добавлено: P2-3, size=2
 Добавлено: P1-4, size=3
 Очередь полна, ждем... (size=3)
 Производитель P1 завершил работу
 Забрано: P1-3, size=2
 Потребитель C1 обработал: P1-3
 Забрано: P2-3, size=1
 Потребитель C2 обработал: P2-3
 Добавлено: P2-4, size=2
 Производитель P2 завершил работу
 Забрано: P1-4, size=1
 Потребитель C1 обработал: P1-4
 Забрано: P2-4, size=0
 Потребитель C2 обработал: P2-4
 Потребитель C1 завершил работу
 Потребитель C2 завершил работу

Когда использовать Condition?

Сценарий	Почему Condition
Producer-Consumer	Нужно ждать, пока появится элемент
Банковские операции	Ждем, пока баланс достигнет нужного значения
Пул соединений	Ждем, пока освободится соединение
Очередь задач	Ждем, пока появится работа

Barrier — точка синхронизации для N корутин. Все ждут, пока не соберётся нужное число участников, затем все продолжают одновременно.

Barrier (барьер) — это примитив синхронизации, который заставляет **N корутин** дожидаться друг друга в одной точке. Представьте группу туристов:

- Все идут по маршруту.
- Когда один доходит до смотровой площадки — он **ждет остальных**.
- Когда **все** собрались — они **вместе** идут дальше.

Схема работы:

Барьер на N=3 участника:

Время t1: Участник А дошел → ждет
Участник В дошел → ждет
Участник С дошел → ждет

Время t2: ВСЕ СОБРАЛИСЬ! → все продолжают одновременно

In [36... *#Базовый API:*

```
# Создаем барьер на 3 участника
barrier = asyncio.Barrier(3)

# В каждой корутине:
async def worker():
    # ... делаем что-то ...
    await barrier.wait() # ждем остальных
    # ... продолжаем все вместе ...

# Особенности:
# - barrier.parties → количество участников
# - barrier.n_waiting → сколько уже ждет
# - barrier.broken → барьер сломан (если одна корутина упала)
# - barrier.reset() → сбросить в исходное состояние
```

In [41... *# Пример 1: Базовое использование*

```
async def runner(name, barrier, delay):
    # Имитация разных скоростей
    await asyncio.sleep(delay)
    print(f"[{time.perf_counter():.2f}]💎 {name}: Добежал до барьера (через {delay})")

    # Ждем всех
    await barrier.wait()

    # Все одновременно продолжают!
    print(f"[{time.perf_counter():.2f}]💎 {name}: Все собрались, бежим дальше!")

async def main():
```

```

barrier = asyncio.Barrier(3) # ждем 3 участников

# Запускаем бегунов с разной скоростью
await asyncio.gather(
    runner("Алиса", barrier, 1),
    runner("Боб", barrier, 3),
    runner("Карл", barrier, 2),
)

await main()

```

```

[22589.07]◇ Алиса: Добежал до барьера (через 1с)
[22590.07]◇ Карл: Добежал до барьера (через 2с)
[22591.08]◇ Боб: Добежал до барьера (через 3с)
[22591.08]◇ Боб: Все собрались, бежим дальше!
[22591.08]◇ Алиса: Все собрались, бежим дальше!
[22591.08]◇ Карл: Все собрались, бежим дальше!

```

In [42... *# Пример 2: Реальная задача – параллельная обработка с синхронизацией*

```

import random

class DataProcessor:
    def __init__(self, num_workers):
        self.barrier = asyncio.Barrier(num_workers)
        self.data = []
        self.results = []

    async def worker(self, worker_id, data_chunk):
        # Этап 1: Загрузка данных
        print(f"W{worker_id}: Загружаю данные...")
        await asyncio.sleep(random.uniform(0.5, 1.5))
        processed = [x * 2 for x in data_chunk]
        self.data.extend(processed)
        print(f"W{worker_id}: Данные загружены")

        # Барьер 1: Все загрузили данные
        await self.barrier.wait()
        print(f"W{worker_id}: Все загрузили, переходим к этапу 2")

        # Этап 2: Агрегация (все работают с общими данными)
        await asyncio.sleep(random.uniform(0.3, 1.0))
        result = sum(self.data) * (worker_id + 1)
        self.results.append(result)
        print(f"W{worker_id}: Результат = {result}")

        # Барьер 2: Все вычислили результат
        await self.barrier.wait()
        print(f"W{worker_id}: Все вычислили, финиш!")

        return result

    async def main():
        processor = DataProcessor(3)

```

```

# Данные для каждого воркера
chunks = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

# Запускаем воркеров
results = await asyncio.gather(*[
    processor.worker(i, chunks[i])
    for i in range(3)
])

print(f"\nИтоговые результаты: {results}")

await main()

```

```

W0: Загружаю данные...
W1: Загружаю данные...
W2: Загружаю данные...
W2: Данные загружены
W0: Данные загружены
W1: Данные загружены
W1: Все загрузили, переходим к этапу 2
W2: Все загрузили, переходим к этапу 2
W0: Все загрузили, переходим к этапу 2
W2: Результат = 270
W0: Результат = 90
W1: Результат = 180
W1: Все вычислили, финиш!
W2: Все вычислили, финиш!
W0: Все вычислили, финиш!

```

Итоговые результаты: [90, 180, 270]

Когда использовать Barrier?

Сценарий	Пример
Многоэтапная обработка	Все воркеры должны завершить этап 1, прежде чем начать этап 2
Распределенные вычисления	Reduce-операция: все собрали данные -> агрегация
Тестирование	Все клиенты одновременно стартуют запрос
Игры	Все игроки готовы -> старт раунда
ML-обучение	Все воркеры загрузили батч -> обновление весов

2.5.4. Проблема «голодания» и справедливости

`asyncio.Lock` по умолчанию использует **FIFO-очередь** ожидающих

корутин. Это значит, что если корутина A захватила lock, а затем B, C, D встали в очередь — они получат lock в порядке B, C, D.

Но в некоторых реализациях (или при ручном написании lock'ов) возможно **голодание (starvation)**: одна корутина постоянно получает lock, а другие ждут бесконечно.

2.5.5. Математическая подоплека: семафоры Дейкстры и deadlock

Семафор был изобретён Эдсгером Дейкстрой в 1965 году. Формально:

- Семафор — это целочисленная переменная $S \geq 0$ с двумя атомарными операциями:
 - $P(S)$ (proberen, проверить): если $S > 0$, декрементировать и продолжить; иначе ждать.
 - $V(S)$ (verhogen, увеличить): инкрементировать S , возможно пробудить ожидающий процесс.

`asyncio.Lock` — это семафор Дейкстры с $S \in \{0, 1\}$ (бинарный семафор, мьютекс).

Deadlock — состояние, когда две или более корутин бесконечно ждут друг друга:

```
In [ ]: lock_a = asyncio.Lock()
lock_b = asyncio.Lock()

async def coro_1():
    async with lock_a:
        await asyncio.sleep(0)
        async with lock_b: # ждёт, пока coro_2 освободит lock_b
            pass

async def coro_2():
    async with lock_b:
        await asyncio.sleep(0)
        async with lock_a: # ждёт, пока coro_1 освободит lock_a
            pass
```

Решения:

- Всегда захватывать lock'и в **одинаковом порядке**.
- Использовать `asyncio.timeout` на блокировках.
- Использовать **иерархические lock'и** (не захватывать lock более

низкого уровня, удерживая lock высокого).

Livelock — корутины не блокированы, но бесконечно «уступают» друг другу, не делая полезной работы. Например, две корутины пытаются захватить lock, видят, что он занят, уступают управление, и так по кругу.

2.6. Потоки и процессы в гибридной модели

2.6.1. Проблема: CPU-bound задачи блокируют event loop

Асинхронность отлична для I/O. Но что если в корутине нужно выполнить тяжёлое вычисление?

```
In [ ]: async def handler():  
        # Это заблокирует весь event loop на секунды!  
        result = heavy_matrix_computation(data)  
        return result
```

Пока `heavy_matrix_computation` выполняется, event loop не может переключиться на другие задачи. Все запросы встают.

2.6.2. `loop.run_in_executor()`: `ThreadPoolExecutor`

Для **блокирующих, но не CPU-интенсивных** задач (например, синхронная библиотека без async API) используется пул потоков:

```
In [ ]: import concurrent.futures  
  
def sync_function(x):  
    # Синхронная функция, которую нельзя переписать  
    return x ** 2  
  
async def main():  
    loop = asyncio.get_running_loop()  
  
    # Выполняем в пуле потоков  
    with concurrent.futures.ThreadPoolExecutor() as pool:  
        result = await loop.run_in_executor(pool, sync_function, 42)  
        print(result)
```

По умолчанию `run_in_executor` использует `ThreadPoolExecutor` с 5 потоками $\times$ число CPU.

Важно: потоки в Python не дают параллелизма для Python-кода из-за GIL. Но они полезны для:

- Вызовов C-библиотек, которые освобождают GIL (`numpy` , `pandas` , `requests` при ожидании сети).
- Блокирующих системных вызовов.

2.6.3. ProcessPoolExecutor: настоящий параллелизм

Для **CPU-bound** задач (ML inference, обработка изображений, криптография) нужны **процессы**:

```
In [ ]: import concurrent.futures

def cpu_intensive(n):
    # Тяжёлое вычисление
    return sum(i * i for i in range(n))

async def main():
    loop = asyncio.get_running_loop()

    with concurrent.futures.ProcessPoolExecutor() as pool:
        # Задача отправляется в отдельный процесс
        result = await loop.run_in_executor(pool, cpu_intensive, 10_000_000)
    print(result)
```

Что происходит:

1. `ProcessPoolExecutor` создаёт пул процессов-воркеров.
2. Функция и аргументы **сериализуются** через `pickle` и отправляются воркеру.
3. Воркер выполняет вычисление в отдельном процессе (свой GIL, свои ядра CPU).
4. Результат сериализуется обратно и передаётся в event loop.

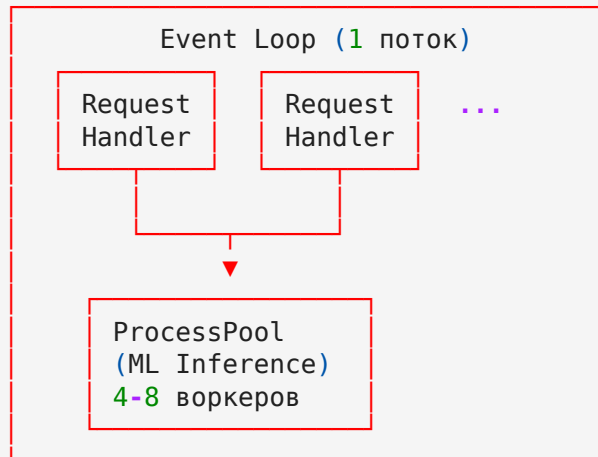
Накладные расходы:

- Сериализация `pickle` для больших данных (например, тензор изображения) может быть дорогой.
- Создание процесса — сотни миллисекунд. Пул процессов амортизирует это.

2.6.4. Паттерн «основной цикл + пул воркеров» для ML

Рекомендуемая архитектура для ML-сервиса на FastAPI:

In []:



- FastAPI обрабатывает I/O в event loop (чтение запроса, валидация Pydantic, запись ответа).
- Тяжёлый inference отправляется в `ProcessPoolExecutor`.
- Результат возвращается в корутину, которая формирует HTTP-ответ.

In []:

```
import asyncio
from concurrent.futures import ProcessPoolExecutor
import numpy as np

# Глобальная модель (загружается один раз в каждом процессе-воркере)
_model = None

def init_worker():
    global _model
    _model = load_ml_model()

def predict(features: np.ndarray) -> np.ndarray:
    return _model.predict(features)

# Создаём пул с инициализацией воркеров
_executor = ProcessPoolExecutor(
    max_workers=4,
    initializer=init_worker
)

@app.post("/predict")
async def predict_endpoint(data: InputModel):
    features = np.array(data.features)
```

```
# Отправляем в пул процессов
loop = asyncio.get_running_loop()
result = await loop.run_in_executor(_executor, predict, features)
return {"prediction": result.tolist()}
```

2.7. Асинхронные генераторы и контекстные менеджеры

2.7.1. Асинхронные генераторы: `async for`

Если генератор должен `await` внутри себя (например, читать данные из сети порциями), используется **асинхронный генератор**:

```
In [ ]: async def fetch_pages(url):
        page = 1
        while True:
            data = await aiohttp.get(f"{url}?page={page}")
            if not data:
                break
            yield data
            page += 1
```

Использование:

```
In [ ]: async for page in fetch_pages("https://api.example.com/items"):
        process(page)
```

Протокол асинхронного генератора:

Метод	Назначение
<code>__aiter__(self)</code>	Возвращает асинхронный итератор (обычно <code>self</code>).
<code>__anext__(self)</code>	Асинхронно возвращает следующий элемент. Выбрасывает <code>StopAsyncIteration</code> , когда закончилось.

Под капотом `async for` работает так:

```
In [ ]: async_iter = fetch_pages(url).__aiter__()
        while True:
            try:
                page = await async_iter.__anext__()
                process(page)
            except StopAsyncIteration:
                break
```

2.7.2. Асинхронные контекстные менеджеры: `async with`

Для ресурсов, у которых инициализация и освобождение асинхронны:

```
In [ ]: from contextlib import asynccontextmanager

@asynccontextmanager
async def database_transaction():
    conn = await pool.acquire()
    tx = conn.transaction()
    await tx.start()
    try:
        yield conn # <- вход в блок with
    except Exception:
        await tx.rollback()
        raise
    else:
        await tx.commit()
    finally:
        await pool.release(conn)

# Использование
async with database_transaction() as conn:
    await conn.execute("INSERT INTO ...")
    # При выходе: commit или rollback + release
```

Протокол:

Метод	Назначение
<code>__aenter__(self)</code>	Асинхронная инициализация. Возвращает объект для <code>as</code> .
<code>__aexit__(self, exc_type, exc_val, exc_tb)</code>	Асинхронное освобождение.

2.7.3. Потокковая обработка и `backpressure`

Асинхронные генераторы часто используются для **потокковой обработки** (streaming). Но здесь кроется опасность: если производитель (generator) генерирует данные быстрее, чем потребитель (consumer) их обрабатывает — происходит **переполнение памяти**.

Это проблема **backpressure** (обратного давления). Решения:

1. **Буферизация с ограничением**: использовать `asyncio.Queue(maxsize=N)`.

2. **Потоковая передача без буферизации:** потребитель запрашивает следующий элемент только когда готов.

Асинхронный генератор естественно реализует второй подход:

`__anext__` вызывается только тогда, когда потребитель готов к следующей порции.

```
In [ ]: async def producer(queue: asyncio.Queue):
        async for item in fetch_stream():
            await queue.put(item)  # блокируется, если queue полна

async def consumer(queue: asyncio.Queue):
    while True:
        item = await queue.get()
        await process(item)
        queue.task_done()

async def main():
    queue = asyncio.Queue(maxsize=100)  # <- backpressure
    await asyncio.gather(producer(queue), consumer(queue))
```

Итог модуля 2

Концепция	Суть	Ключевой API
Конкурентность	Перекрывающееся выполнение задач в одном потоке	<code>async / await</code>
Параллелизм	Одновременное выполнение на разных ядрах	<code>ProcessPoolExecutor</code>
Event Loop	Цикл, распределяющий управление между корутинами	<code>asyncio.run()</code> , <code>get_running_loop()</code>
Корутина	Приостанавливаемая функция с сохранением состояния	<code>async def</code>
Task	Корутина, запущенная в event loop	<code>asyncio.create_task()</code>
Future	Контейнер для будущего результата	<code>asyncio.Future()</code>
Gather	Конкурентный запуск с сохранением порядка результатов	<code>asyncio.gather()</code>
Lock	Взаимное исключение для разделяемых ресурсов	<code>asyncio.Lock()</code>
Semaphore	Ограничение числа одновременных операций	<code>asyncio.Semaphore()</code>
Отмена	Прерывание задачи через <code>CancelledError</code>	<code>task.cancel()</code>
Таймаут	Автоматическая отмена по истечении времени	<code>asyncio.timeout()</code>

Концепция	Суть	Ключевой API
Executor	Запуск синхронного/CPU-bound кода вне event loop	<code>run_in_executor()</code>
Async for/with	Асинхронные аналоги итераторов и менеджеров	<code>__aiter__</code> , <code>__aenter__</code>

В **Модуле 3** мы спустимся на уровень сетевых протоколов: разберём TCP, HTTP, WebSocket — чтобы понимать, какие именно I/O-операции оптимизирует event loop.